SilentVLA Research Explanation

Prompt-Induced Action Drift in OpenVLA, Why It Matters, and How This Project Can Grow into a Safer VLA Paper

Extended research draft built from the completed local OpenVLA experiments on BridgeData V2 images.

Harmit Patel

Student ID: 110164115

Department of Computer Science, University of Windsor, Windsor, Ontario, Canada

Supervisor: Sheriff Saad

Abstract

Vision-language-action models promise a simple interface for robotics: the model sees an image, reads an instruction, and predicts the next robot action. That promise is exciting, but it also raises a safety question. What happens if the words change a little while the scene stays the same? In this project I study that question in OpenVLA-7B by running two complementary local experiments on BridgeData V2 images. The first branch, Manual40, uses 40 manually selected trajectories with task-specific prompts written by hand. The second branch, Auto50, uses 50 automatically sampled trajectories from the BridgeData scripted split and prompts built from run-family metadata. For each image, I compare four prompt conditions: normal, paraphrased, contradictory, and neutral. I measure how much the first six action dimensions drift away from the normal prompt using L2 distance and cosine similarity. Across both branches, paraphrases preserve behavior best, while contradictory and neutral prompts usually cause larger action drift. Final-state images are more fragile than initial-state images. These results do not yet prove full task failure in simulation or on a real robot, but they do provide a clear, reproducible warning signal for silent failure risk in VLA systems. The document also explains why OpenVLA was chosen, why a 30 GB BridgeData download was worth doing, how the local WSL pipeline was debugged, how the attention-mask patch was fixed, what succeeded, what failed, and which future work would make this project conference stronger.

TL; DR

Think of OpenVLA as a robot brain that takes two things at once: a camera picture and a sentence. It then decides how the robot arm should move next. This sounds simple, but it hides a real risk. If the words change from “pick up the cup” to “grab the mug,” the robot might still do almost the same thing. That is good. But if the words change to something vague or contradictory, the robot might silently drift toward a different action without saying it is confused. My project checks exactly how often that drift happens. I used real robot images from BridgeData V2 and compared the predicted actions under four prompt types. I found that small friendly rewrites are usually okay, but contradictory and neutral prompts can move the action much more. That matters because future home robots, warehouse robots, and hospital robots will not receive instructions in exactly one perfect sentence every time. They need to stay stable even when people speak differently.

1. Introduction and Motivation

I chose this topic because I wanted to test whether the “robot brain” feeling of modern VLA models is actually reliable, or whether that intelligence becomes fragile when the wording changes. The curiosity started when I was experimenting with OpenVLA and noticed that two slightly different prompts on the same image produced noticeably different action vectors. That moment turned into the research question for this project: how sensitive is OpenVLA to prompt wording, and when does that sensitivity become dangerous?

This question matters in the 2025–2026 VLA literature. OpenVLA itself was introduced as a 7B-parameter open-source VLA trained on 970k real-world robot demonstrations [1]. At nearly the same time, π0 was proposed as a general robot control model built on a pre-trained vision-language model plus a flow-matching action architecture [2]. These systems made the field much more exciting, but they also made a new problem visible: researchers can now run strong open models locally and inspect their failures, rather than treating them as black boxes [1][2].

Recent 2026 papers show that language grounding is a serious open issue for VLAs. LangGap reports that current SOTA(state-of-the-art) VLA models can largely ignore language under carefully designed semantic perturbations [7]. IGAR shows that VLAs may still execute visually plausible actions even when

the instruction is contradictory, a failure mode it calls linguistic blindness [6]. LangForce frames part of the same problem as information collapse, where the contribution of language to action prediction becomes too weak under certain conditions [8]. SAFE and FailSafe further show that the community is now actively studying detection and recovery from VLA failures [9][10]. Diversity-aware red teaming work from 2026 strengthens the same message by showing that linguistic variations can sharply reduce task success in simulation [11].

My project does not claim to solve all of that. It does something narrower but still valuable: it measures prompt-induced action drift in one concrete open model, OpenVLA-7B, on a reproducible subset of BridgeData V2 images. Instead of inventing a new benchmark from scratch, I took a model people can actually run, a dataset people can actually cite, and a local workflow that can be repeated on a student machine. Then I tested four prompt families on both initial and final states. That gives a clean baseline: before building a safety detector, before adding a recovery policy, before porting to π0, how unstable is the model already?

The answer is meaningful. Across both manual and automated branches, paraphrases are usually the least disruptive, while contradictory and neutral prompts produce larger drift. Final-state images are more fragile than initial-state images. That gives a paper story: the problem is real, it is measurable, it appears in both a hand-curated and a metadata-driven branch, and it suggests a future path toward silent-failure anticipation rather than blind trust in language-conditioned policies.

2. Why This Matters in Real Life

Home robots will hear many styles of language. One person may say “put the pot on the towel,” another may say “place the pan on the cloth,” and a third may say something vague like “deal with that pot.” A stable robot should not change behavior dramatically just because the sentence surface changed.

Factory robots often run near people, shelves, tools, and expensive equipment. If language-conditioned control is brittle, a small prompt difference could become a productivity or safety issue rather than a harmless academic artifact.

Hospital, elder-care, and assistive robots need even stronger robustness. A robot that silently follows visual priors while ignoring instruction details could become misleading or unsafe. The 2026 literature already treats language grounding, robustness, and failure anticipation as first-class problems for real deployment [6][7][9][10][11].

That is why this project is useful even before a full intervention is added. It provides a reproducible empirical baseline showing when wording changes are minor and when they are risky enough to deserve a future detector or fallback policy.

3. Background: What Are VLA Models, and Why OpenVLA?

A vision-language-action model, or VLA, is a model that takes vision and language as input and predicts an action for a robot. The “vision” part means images or video from a camera. The “language” part means an instruction such as “pick up the rigid object.” The “action” part means the next motion command for the robot arm or end effector. The recent VLA survey describes this as the unification of perception, language understanding, and embodied action inside one framework [4].

OpenVLA was a natural choice for this project for three reasons. First, it is open-source and widely used in the research community [1]. Second, it was designed to be adaptable and inspectable, which matters if the goal is to analyze behavior rather than merely report a score [1]. Third, it can be run locally with a reasonably careful setup, which made it realistic for a student-led project without depending on a

large lab cluster. In the official paper and project page, OpenVLA is described as a 7B-parameter model trained on 970k real robot episodes from Open X-Embodiment [1].

π0 is also highly relevant. It is built around a flow-matching action model on top of a pre-trained vision-language model and was explicitly designed for general robot control, including dexterous and longer-horizon tasks [2]. In many ways, π0 is a stronger stress-test candidate for a future follow-up paper because it represents a different action architecture than OpenVLA. However, for this project I chose OpenVLA as the primary experimental model because it was easier to instrument, easier to patch locally, and easier to run end-to-end inside the local WSL workflow. That is not a claim that OpenVLA is categorically better than π0. It is a claim about experimental practicality and reproducibility for this project.

Table 1. Why OpenVLA was used now, and where π0 fits next

Dimension	OpenVLA in this project	π0 in future work	Why this matters
Openness and access	Fully open paper, public code, public checkpoints [1]	Public paper and growing ecosystem [2]	Both are research-usable, but OpenVLA was easier to instrument immediately
Architecture style	Autoregressive token-style VLA [1]	Flow-matching VLA [2]	A future comparison would test whether drift patterns depend on the action architecture
Local debugging value	Very high for this project	Moderate to high, but not yet integrated here	This project needed a model that could be patched and rerun quickly
Completed experiments	Yes	No	Important honesty point: π0 is discussed as a comparison target, not a completed experiment

4. The Research Gap This Project Actually Fills

The cleanest research gap for the completed work is not “I solved silent failure.” That would be an overclaim. The gap filled here is more careful: there was no systematic, locally reproducible measurement in this project pipeline of how much OpenVLA action vectors move when prompt wording changes across both manually curated and automatically sampled BridgeData image subsets. In other words, before building a detector or a recovery system, I needed a trustworthy map of prompt sensitivity.

This gap is still worth filling because the broader literature mostly asks bigger questions: do VLAs ignore language [7]? Can train-free attention recalibration repair some contradictions [6]? Can a trained detector identify failures across tasks [9]? Can a recovery model learn to reason about failures and repair them [10]? Those are important questions, but they do not remove the need for a concrete, low-level baseline showing how large the action changes already are in a real open-model workflow.

My contribution is therefore an empirical bridge between broad failure theory and actual local model behavior. It answers practical questions a researcher or student might ask before attempting a safety layer: Are paraphrases mostly harmless? Are contradictory prompts much worse? Are neutral prompts also dangerous because they wash out task information? Are late or near-completion scenes more fragile than early scenes? The results show that the answer is yes to all four, with final-state scenes showing especially elevated drift.

5. Data, Prompt Design, and Why a 30 GB Dataset Download Was Worth It

BridgeData V2 was the right dataset family for this project because it is large, open, and directly relevant to language-conditioned manipulation. The official BridgeData V2 description reports 53,896 trajectories collected across 24 environments on a low-cost robot and emphasizes scalable robot learning with natural language and goal-image conditioning [3]. That scale matters. A tiny toy dataset would have made the paper easy to criticize. Using BridgeData V2 gave the project a legitimate, citable base.

The full scripted subset is large enough to feel inconvenient on a student machine, which is exactly why it matters in a real project story. The 30 GB local download was not done for fun. It was done to move from a toy pilot to a proper sample grounded in an official dataset. Once the data was local, I could build an automated 50-trajectory branch with reproducible naming, metadata, and prompt generation. That is a major quality jump over a handful of random screenshots.

The two experiment branches serve different roles. Manual40 is stronger semantically because the prompts were written from the actual scene and task intention by hand. Auto50 is stronger procedurally because it comes from a larger scripted subset and uses run-family metadata rather than pure manual curation. Together they create a balanced design: one branch is close to what a researcher would hand-build, and the other is closer to what a reproducible benchmark subset would look like.

The prompt design is simple but important. For every image I created four prompt families: normal, paraphrased, contradictory, and neutral. The normal prompt expresses the intended task. The paraphrased prompt keeps the meaning but changes the wording. The contradictory prompt reverses or conflicts with the task intention. The neutral prompt removes task-specific content and asks only for a generic action. This design directly tests whether the model uses language semantically, loosely, or barely at all.

6. Environment, Local WSL Setup, and the Attention-Mask Patch

All completed experiments were run locally, not in a large hosted cluster. The working environment used Ubuntu under WSL2 with a Python virtual environment, a 4-bit loading configuration, and an NVIDIA GeForce RTX 2060 GPU. That matters because part of the value of this project is that it demonstrates a real research workflow on modest hardware rather than assuming large-lab infrastructure.

The engineering story was not smooth. The project hit the common VLA pain points that student researchers actually face: Python environment issues, dependency mismatches, missing packages such as psutil and draccus, model loading friction, and image-path problems. The key technical bug was a one-token attention-mask mismatch after vision fusion in the remote model code. In plain English, the model was building a longer internal sequence once image tokens were added, but the attention mask still matched only the text tokens. That broke generation. The practical fix was to patch predict_action so it prepended an image attention mask before inference. After that, dummy-image tests, real-image tests, and the full drift experiments ran correctly.

This engineering section matters for two reasons. First, it proves the work was real rather than purely theoretical. Second, it makes the project reusable for someone else who wants to build on it. Research is not only about the final number. It is also about whether another person can understand the path from broken setup to stable pipeline.

7. Experimental Protocol

The protocol is deliberately simple. For each image, the normal prompt is treated as the anchor. The model predicts one action vector for the normal prompt and three additional action vectors for the paraphrased, contradictory, and neutral prompts. Only the first six action dimensions are analyzed

numerically because the seventh dimension, the gripper-related term, was either near-constant or not informative enough for the chosen analysis. That is a limitation, but it is an honest and common one in exploratory VLA evaluation.

Two metrics are used. The first is 6D L2 distance between the action under the normal prompt and the action under each perturbed prompt. Higher L2 means larger drift. The second is cosine similarity between the same vectors. Higher cosine means the direction is more preserved. These metrics are simple, understandable, and sufficient for a first pass. They do not claim full task semantics or safety on their own, but they do quantify how strongly the action changes when the wording changes.

The analysis is done separately for initial and final images in both Manual40 and Auto50. That separation turned out to be scientifically useful. The final-state images, where the task is closer to completion, are consistently more fragile than the initial-state images. That would have been hidden if all images were pooled together.

9. How to Read the Findings

The project found four durable patterns. First, paraphrased prompts are usually the least disruptive. This is good news because it suggests OpenVLA has some degree of robustness to harmless rewording. Second, contradictory prompts often increase drift sharply, which matches the broader literature on linguistic blindness and information collapse [6][7][8]. Third, neutral prompts are not safe by default. In many cases, removing task detail lets the model drift away from the intended action almost as much as a contradictory instruction. Fourth, final-state images are more fragile than initial-state images. That is one of the most interesting results in the whole project because it suggests that near-completion states may be more semantically brittle or more underdetermined.

One reasonable interpretation is that early-stage scenes provide more spatial freedom. The model can still move in several broadly valid directions and remain semantically aligned. Late-stage scenes may require more precise language grounding because the next move depends on subtle task context: the object may already be near the target, partially manipulated, or close to a failure boundary. If the prompt becomes vague at that point, the model may “fill in” the missing instruction with visual priors or generic motion tendencies. That fits the 2026 narrative in IGAR, LangGap, and LangForce: language is not always used as strongly as benchmark success rates might suggest [6][7][8].

10. Where This Project Succeeded

This project succeeded in ways that matter for real research: it established a working local OpenVLA pipeline on modest hardware; it fixed a real inference bug; it completed two complementary experiment branches rather than stopping at a tiny pilot; it produced clear and stable numerical patterns rather than ambiguous noise; it generated reusable metadata, CSV results, summary tables, and failure-case lists; and it created a strong base for a future silent-failure detector paper, because the drift signal is now measured rather than guessed.

11. Where This Project Failed or Stayed Incomplete

The project did not yet close the loop with a simulator outcome metric such as task success or collision rate. It did not yet run π0 or another second VLA as a cross-model ablation. It did not yet test an intervention such as re-prompting, abstention, or active visual recovery. The seventh action dimension was not used in the main analysis because it did not behave helpfully enough for the chosen metrics. The Auto50 prompts were derived from run-family metadata, which is reproducible but semantically weaker

than careful hand writing. The project also used static images, not full rollout dynamics, so it measures action drift, not full embodied success.

12. Why the Problem Is Still Not Solved

This problem is still open because VLA robustness is hard at several levels at once. First, the model internals are complex, and extracting stable cross-modal signals from them without retraining is difficult. SAFE addressed failure detection with a trained detector [9]. IGAR addressed contradictory instruction handling with train-free attention recalibration [6]. FailSafe addressed recovery through generated failure and repair data [10]. Each of those tackles a slice of the problem, but none eliminates the need for careful, model-specific empirical measurement.

Second, benchmarks themselves are evolving. BridgeData V2 is large and practical [3], LIBERO is standard for manipulation evaluation, and LIBERO-X, LangGap, and DAERT all show that robustness testing needs better semantic stress cases [5][7][11]. That means the target keeps moving: as soon as one narrow weakness is found, a new benchmark can expose the next one.

Third, intervention work is expensive. A true “silent failure” paper usually wants at least three pieces: a detector, a response policy, and an outcome study showing that the response improves success or safety. My completed work fully supports the detector motivation, but it stops one layer earlier by providing the drift evidence and pipeline needed before such a detector can be evaluated honestly.

13. OpenVLA and π0: Honest Comparison for a Stronger Future Paper

A stronger future paper version would compare OpenVLA and π0 directly. That is worth doing because the two models differ in architecture and therefore may differ in how wording changes appear inside action space. OpenVLA is an obvious baseline because it is open, popular, and easy to instrument [1]. π0 is a strong comparison target because it was designed as a general robot control model with a different action-generation strategy [2]. If both models show the same prompt-drift ranking, then the result is more likely to reflect a field-wide VLA property rather than one model quirk. If they differ, that is also valuable because it suggests the failure mode depends on architecture.

14. Practical Applications

Home robotics: prompts from children, parents, elders, or non-native speakers will vary in wording. Drift measurement helps estimate how robust the robot is before it enters the home.

Industrial robotics: task wording in human-robot collaboration may be shortened, paraphrased, or context-dependent. Drift analysis can reveal when a system needs instruction confirmation.

Healthcare and assistive systems: language-conditioned motion should be stable, especially near task completion, where my results suggest fragility is highest.

Robotics education and benchmarking: the local OpenVLA workflow provides a repeatable path for students to evaluate robustness without needing a huge lab cluster.

15. The Best Next Research Move: SilentVLA-Style Failure Anticipation

The most defensible next paper is not to pretend that the current project already solved silent failure. The right move is to build on these completed results and propose a narrow, training-free failure anticipation layer for a frozen VLA. The idea would be simple: monitor cross-modal collapse signals such as action entropy, language-attention drift, action repetition, or a lightweight mutual-information proxy; if

those signals rise above a threshold, trigger a minimal response such as abstention, re-prompting, or a safe re-observe motion. That direction matches the momentum of the recent literature without copying any one existing method exactly [6][8][9][10][11].

My current results help justify that direction because they already show where the detector should work hardest: contradictory and neutral prompts, especially on final-state images. In other words, the present project provides the measurement layer that a future SilentVLA paper would need before claiming a safety benefit.

16. Future Work, What Else Could Be Done, and What Stops It Right Now

Outcome-aware simulation study: replay or simulate the highest-drift cases in a robotics benchmark to test whether action drift turns into real task failure. This is not solved yet because it requires careful environment binding between the image-level experiment and a task-level simulator.

Training-free silent-failure detector: build a score from action entropy, language-attention drift, and action repetition. This is not solved yet because VLA internals are hard to extract consistently across models and tasks.

Lightweight recovery policy: safe stop, re-observe, or re-prompt when the detector fires. This is not solved yet because it requires a task outcome loop, not only a static image loop.

Cross-model validation on π0 or another VLA: this is blocked mainly by time, engineering effort, and the need to re-create the instrumentation path for another model family.

Trajectory-level evaluation instead of single-image inference: this is blocked by compute cost, simulator integration, and the extra complexity of measuring task success rather than vector drift.

Better semantics for Auto50 prompts: this is limited by the dataset metadata itself. Run-family text is useful but not as rich as hand-written prompt descriptions.

Real-robot verification: this is blocked by hardware access, deployment risk, and the need for additional safety layers before physical testing.

17. A Step-by-Step Guide for Someone Starting from Zero

If I had to explain this project to a complete beginner, I would say this: first, pick one open robot brain you can actually run. In this project that was OpenVLA. Second, pick one real dataset you can cite. In this project that was BridgeData V2. Third, create a small but serious subset, not a fake toy demo. Fourth, test multiple prompt types on the same image. Fifth, measure how much the action changes. Sixth, interpret the result honestly. That is the whole project skeleton.

The local execution path was: install the Python environment, make sure the GPU was visible, load OpenVLA in 4-bit mode, fix the attention-mask bug, run a dummy-image test, run a real-image test, build metadata for Manual40, build metadata for Auto50, run the four prompt families, summarize the results, and then write the paper around what actually happened instead of what I wished had happened.

A beginner-friendly way to think about the metrics is also simple. L2 distance asks, “How far apart are the two action vectors?” Cosine similarity asks, “Are they still pointing in the same direction?” If paraphrase stays close while contradiction and neutrality move away, then the model has at least some useful language sensitivity. If everything stays the same, the model may be ignoring language. If everything changes chaotically, the model may be fragile.

18. My OpenVLA Journey - Documentary in Written Form

I got interested in this topic because robots felt magical and suspicious at the same time. A robot that can look at a picture and read a sentence sounds like science fiction. But the moment something feels magical, I want to know where it breaks. That is what led me here.

The idea became real when I was playing with OpenVLA and saw action numbers change more than I expected under different prompt wordings. That small surprise grew into a research question. I did not start by saying, “I will solve VLA safety.” I started by saying, “Why did the action move that much when the picture did not change?”

Then the messy real research part began. Dependencies were missing. The virtual environment failed at first. The model downloaded but inference crashed. The remote model code brought a token mismatch. The image paths were wrong. The data organization was messy. There were moments when it looked easier to give up and write a smaller report. But the point of research is not to stop at the first ugly obstacle. The point is to make the system work well enough that the question becomes answerable.

One important turning point was deciding not to stay with a tiny pilot. I wanted something conference-respectable, so I downloaded the larger BridgeData subset and built the automated branch. That was the moment the project stopped being a toy curiosity and started looking like real work. Manual40 gave me semantic control. Auto50 gave me scale and reproducibility. Together, they turned the story into evidence.

What I discovered is simple to say and hard to ignore: small wording changes matter. Friendly paraphrases are usually okay. Contradictions and vague instructions are much riskier. Final-state scenes are extra risky. That means the robot brain is not just reading language in one stable, perfect way. It is balancing vision and language in a way that can drift under prompt pressure.

Why is that important in real life? Because future robots will not live inside one clean benchmark sentence. They will live with real people, messy language, incomplete instructions, noise, and changing scenes. If the robot silently drifts when the wording changes, then safety and trust become real issues, not abstract ones.

What I learned about research is that honest debugging is part of the science. Research is not just the final graph. It is the full path: the bad installs, the patch, the moment of doubt, the small victory when the first real image works, and the deeper victory when the results tell a consistent story. I also learned that good research questions often begin as a simple annoyance you cannot ignore.

My future plan is clear. I want to turn this drift study into a stronger silent-failure paper with a small inference-time detector and a real task-level evaluation. I also want to test a second model such as π0 so the findings become more general. This project is not the end of the story. It is the clean first chapter.

19. Conclusion

This project completed a real, local, reproducible study of prompt-induced action drift in OpenVLA-7B on BridgeData V2 images. It showed that paraphrases are usually the safest prompt perturbation, contradictory and neutral prompts cause larger action drift, and final-state scenes are more fragile than initial-state scenes. It also documented the engineering path needed to make such a study run on modest hardware. The work does not yet solve silent failure, but it does something necessary before that solution can be claimed: it measures the problem carefully, transparently, and in a way another researcher can build on. As a master draft, this document is designed to be readable by a beginner, useful to a professor, and compressible into a future conference paper.

References

I would suggest everyone to read these papers and read blogs on robotics whenever you can.

[1] Kim et al., OpenVLA: An Open-Source Vision-Language-Action Model, PMLR 2025 / arXiv 2024.

[2] Black et al., π0: A Vision-Language-Action Flow Model for General Robot Control, arXiv 2024.

[3] Walke et al., BridgeData V2: A Dataset for Robot Learning at Scale, CoRL 2023 / OpenReview.

[4] Sapkota et al., Vision-Language-Action Models: Concepts, Progress, Applications and Challenges, arXiv survey 2025/2026.

[5] Wang et al., LIBERO-X: Robustness Litmus for Vision-Language-Action Models, arXiv 2026.

[6] Zhang et al., Restoring Linguistic Grounding in VLA Models via Train-Free Attention Recalibration (IGAR), arXiv 2026.

[7] Hou and Zhao, LangGap: Diagnosing and Closing the Language Gap in Vision-Language-Action Models, arXiv 2026.

[8] Lian et al., LangForce: Bayesian Decomposition of Vision Language Action Models via Latent Action Queries, arXiv 2026.

[9] Gu et al., SAFE: Multitask Failure Detection for Vision-Language-Action Models, arXiv 2025.

[10] Lin et al., FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models, arXiv 2025.

[11] Tong et al., Uncovering Linguistic Fragility in Vision-Language-Action Models via Diversity-Aware Red Teaming, arXiv 2026.

[12] CoRL 2026 Instruction for Authors and Call for Papers, official venue website and OpenReview 2026 submission page.

Appendix A. Detailed Related-Work Notes

OpenVLA matters in this project because it lowered the barrier to entry for VLA research. Before open models, many researchers could discuss VLAs in theory but could not inspect a full checkpoint and patch its inference stack themselves. That openness is part of the reason this project exists at all. A student can now move from reading a paper to running the model locally, which changes the type of empirical question that is realistic to ask [1].

π0 matters for a different reason. It represents a strong alternative design for general robot control and is already treated as a serious baseline in the new robustness and grounding papers. That makes π0 the correct second model for a future follow-up, especially if the goal is to test whether prompt-induced drift is an OpenVLA-specific phenomenon or a more general VLA behavior [2].

BridgeData V2 matters because it gives this project real images from a citable manipulation dataset rather than synthetic scenes invented only for the paper. That choice increases external validity. It does not solve all realism questions, but it is much stronger than building a paper on a tiny custom folder of screenshots [3].

LIBERO-X, LangGap, IGAR, SAFE, FailSafe, and linguistic-fragility red teaming all point in the same broad direction: current VLAs are not as language-robust as their headline success rates might make people think. What differs between them is which layer of the problem they attack. Some diagnose, some recalibrate, some train a detector, some generate recovery data, and some systematically search for adversarial instructions. This project sits one layer below them by asking a simpler question first: before adding any intervention, how much does the action actually move when the words change? [5][6][7][9][10][11]

Appendix B. Plain-English Glossary

- Action vector: a list of numbers that tells the robot how to move next. In this project, I mainly analyzed the first six movement numbers.
- BridgeData V2: a large robot-learning dataset with many manipulation trajectories collected in different environments [3].
- Contradictory prompt: a sentence that conflicts with the intended task, such as telling the robot to avoid the target instead of moving toward it.
- Cosine similarity: a number that tells whether two vectors still point in roughly the same direction. Closer to 1 means more aligned.
- Final-state image: an image taken near the end of a task, when the next move may need more precise language grounding.
- Information collapse: a recent idea that the language signal can become too weak compared with the visual signal, so the action is driven mainly by the image [8].
- Initial-state image: an image taken near the beginning of a task, when the robot usually has more possible valid moves.
- L2 distance: a number that tells how far apart two vectors are. Bigger means more drift.
- Neutral prompt: a generic instruction that removes the task-specific meaning and asks for some action without enough detail.
- OpenVLA: an open-source vision-language-action model used in this project [1].
- Paraphrased prompt: a sentence that keeps the same meaning but changes the wording.
- π0: another VLA-style robot control model used here as a future comparison target [2].
- Prompt-induced action drift: the change in predicted robot action when the wording changes but the image stays the same.
- Silent failure: a risky situation where the model starts doing the wrong thing without clearly signaling that it is confused. This project motivates future silent-failure work but does not claim to solve it yet.
- WSL2: Windows Subsystem for Linux, the local environment used to run the model pipeline on the laptop.

Appendix C. Extended Discussion of OpenVLA Versus π0

OpenVLA and π0 are both worth attention because they stand for two different styles of “general robot policy.” OpenVLA is especially attractive when the goal is openness, inspectability, and community reproducibility. π0 is especially attractive when the goal is to test whether a different action-generation design changes robustness. A good long draft should explain both, because doing so shows that the project is not stuck inside one narrow tool choice.

From the point of view of this project, OpenVLA offered a lower-friction path to a first complete result. That matters more than people sometimes admit. In robotics, the best theoretical idea can still fail as a project if the first baseline is too hard to instrument. Choosing an experimentally tractable model is not a weakness. It is often the reason the first publishable measurement exists at all.

At the same time, a future π0 comparison would do more than just decorate the paper. If a drift pattern holds across OpenVLA and π0, then the result gains external credibility. If the models disagree, then that disagreement itself becomes an insight about architecture and language grounding. Either outcome is useful.

Appendix D. Expanded Limitations

- This project measures action drift, not actual damage, collisions, or task completion. That means it is a risk indicator, not a full safety guarantee.
- The prompts are English-only and relatively short. A multilingual or dialog-style study might show different behavior.
- The project uses single images rather than full temporal rollouts. Real robots act over time, so a full deployment study could strengthen or weaken some conclusions.
- The neutral prompt family is intentionally generic, but there are many ways to define “neutral.” Another neutral prompt design might shift the exact numbers.
- The manual branch is semantically stronger, while the auto branch is procedurally stronger. That balance is useful, but it also means the two branches are not identical in data-creation style.
- The local GPU and 4-bit configuration made the project feasible, but some very fine-grained timing or hidden-state analyses might behave differently on other hardware stacks.

Appendix E. In short summary of this experiment

This project is about checking whether a robot brain stays calm when the words change. I kept the same picture and changed only the sentence. Then I checked whether the robot’s next move stayed similar or drifted away.

I found that friendly rewrites are mostly okay, but wrong or vague wording can push the action much further away. I also found that scenes near the end of a task are more fragile than scenes at the start.

So the message is simple: if we want trustworthy robots, we cannot only ask whether they can do the task. We also must ask whether they still do the right thing when people speak in slightly different ways.

Appendix F. Long-Form Reflection on Why This Project Matters to Me

This appendix is intentionally personal and intentionally long. I am keeping it because research is not only a list of numbers. In practice, a project becomes meaningful when the researcher can explain why it mattered enough to finish. For me, this project mattered because it turned a loose curiosity about robot intelligence into something testable. Instead of just watching robot videos and thinking they looked cool, I learned how to ask a concrete question, collect evidence, debug a broken stack, and end with a result that says something specific about real models.

It also mattered because the project forced me to see how modern robotics research actually works. The popular image of robotics is that a person writes one script and the robot suddenly works. The real image is different. The real image is repeated setup, repeated failure, repeated repair, and then a slowly growing sense that the pipeline is finally stable enough to answer the question honestly. That process

changed the way I think about research. It made me more comfortable with not knowing the answer at the start.

Finally, this project matters because it sits at the edge between software and physical safety. A wording change may sound small in software terms, but in robotics it can become a motion change. That connection is exactly why I think this line of work is useful. Even when the final result is “I measured the problem but did not solve it yet,” that measurement still helps the field move toward safer robots. That is a contribution worth making.